Solving the Traveling Salesman Problem Using Ant Colony Optimization

Hothifa Hamdan[1], Mazen Khaled[1], Ahmed Ayman[1], Hager Ali[1]

***Abstract*— The Traveling Salesman Problem (TSP) represents a cornerstone in combinatorial optimization, aiming to find the shortest Hamiltonian circuit in a weighted graph. Ant Colony Optimization (ACO), a bio-inspired metaheuristic algorithm, mimics the foraging behavior of ants to solve complex optimization problems effectively. This report implements and analyzes ACO for solving TSP, comparing its performance with Genetic Algorithms (GA) and Sequential Quadratic Programming (SQP). Comprehensive experimental results demonstrate ACO's superior solution quality, while GA and SQP offer faster computational times. Parameter sensitivity analysis further highlights the critical role of pheromone influence and heuristic information in optimizing ACO's performance. The findings provide valuable insights into algorithm selection criteria and implementation considerations for tackling NP-hard optimization problems.**

I. INTRODUCTION

The **Traveling Salesman Problem (TSP)** seeks the shortest tour for a salesman visiting all cities exactly once and returning to the starting point. Due to its NP-hard nature, it serves as a benchmark for testing optimization algorithms [1], [2].

Ant Colony Optimization (ACO) is a bio-inspired algorithm introduced by Marco Dorigo in the 1990s. It models artificial ants that iteratively construct solutions, leveraging pheromone trails and heuristic information to balance exploration and exploitation in searching for optimal solutions [3].

The primary objective of this report is to implement ACO for solving TSP and compare its effectiveness against other optimization techniques such as Genetic Algorithms (GA) and Sequential Quadratic Programming (SQP). The comparison focuses on solution quality, computational efficiency, and parameter sensitivity.

II. THEORETICAL BACKGROUND

A. Traveling Salesman Problem (TSP)

The TSP can be formally defined as follows:

$$
\begin{aligned}
\text{Minimize} \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} d_{ij}x_{ij} \\
\text{Subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1 \quad \forall i \\
& \sum_{i=1}^{n} x_{ij} = 1 \quad \forall j \\
& \sum_{i\in S}\sum_{j\in S} x_{ij} \le |S|-1 \quad \forall S \subset V, 2 \le |S| \le n-1 \\
& x_{ij} \in \{0,1\} \quad \forall i,j
\end{aligned}
\tag{1}
$$

Where:

- n is the number of cities.
- d_{ij} represents the distance between city i and city j.
- x_{ij} is a binary variable indicating whether the path from city i to city j is included in the tour.

B. Ant Colony Optimization (ACO)

ACO is inspired by the foraging behavior of real ants. In nature, ants communicate indirectly through pheromone trails to find the shortest path between their colony and a food source. This collective behavior allows ants to optimize their search without centralized control.

1) ACO Components:

- **Pheromone Trails**: Represented by τ_{ij}, indicating the desirability of moving from city i to city j.
- **Heuristic Information**: Represented by η_{ij}, typically the inverse of the distance between cities ($\eta_{ij} = 1/d_{ij}$).
- **Ants**: Artificial agents that construct solutions by traversing the cities based on pheromone levels and heuristic information.
- **Pheromone Update Rules**: Determine how pheromone levels are updated based on the quality of solutions found.

2) ACO Probability Function: The probability that an ant moves from city i to city j is given by:

$$
P_{ij} = \frac{(\tau_{ij})^{\alpha}(\eta_{ij})^{\beta}}{\sum_{k\in N_i}(\tau_{ik})^{\alpha}(\eta_{ik})^{\beta}} \tag{2}
$$

Where:

- α controls the influence of pheromone trails.
- β controls the influence of heuristic information.
- N_i is the set of cities not yet visited by the ant.

*This work was supported by the University of Science and Technology in Zewail City.

[1]Hothifa Hamdan, Mazen Khaled, Ahmed Ayman, and Hager Ali are with the Department of Data Science and Artificial Intelligence, School of Computer Science and Artificial Intelligence, University of Science and Technology in Zewail City, Egypt. Emails: s-hothifa.mohamed@zewailcity.edu.eg, s-mazen.ahmed@zewailcity.edu.eg, s-ahmed.barakat@zewailcity.edu.eg, s-hager.abdelaziz@zewailcity.edu.eg

C. Genetic Algorithms (GA)

Genetic Algorithms are inspired by the process of natural selection. They operate on a population of candidate solutions, applying operators such as selection, crossover, and mutation to evolve solutions over generations.

1) GA Components:

- **Population**: A set of individuals (candidate solutions).
- **Fitness Function**: Evaluates the quality of each individual.
- **Selection**: Chooses individuals based on fitness to breed the next generation.
- **Crossover**: Combines parts of two parents to produce offspring.
- **Mutation**: Introduces random changes to offspring to maintain diversity.

D. Sequential Quadratic Programming (SQP)

SQP is an iterative method for nonlinear optimization. It solves a sequence of optimization subproblems, each approximating the original problem using a quadratic model.

1) SQP Components:

- **Quadratic Subproblems**: Each subproblem approximates the original problem using a quadratic model of the objective function and linearizes the constraints.
- **Hessian Matrix**: Represents the second-order derivatives of the objective function.
- **Line Search**: Determines the step size to ensure convergence.

E. Complexity Analysis

- **TSP**: NP-hard problem with factorial time complexity for exact solutions.
- **ACO**: $\mathcal{O}(n^2 m)$ per iteration, where n is the number of cities and m is the number of ants.
- **GA**: $\mathcal{O}(gps^2)$ where g is generations, p is population size, and s is solution size.
- **SQP**: $\mathcal{O}(n^3)$ per iteration for quadratic subproblems.

III. METHODOLOGY

A. Ant Colony Optimization Algorithm

The ACO algorithm is structured as follows:

1) **Initialization**:
   - Initialize pheromone levels τ_{ij} uniformly across all edges.
   - Define ACO parameters: α, β, ρ, and Q.
   - Place ants randomly at different starting cities.
2) **Tour Construction**:
   - Each ant constructs a tour by moving from city to city based on the probability defined in Equation 2.
   - The selection balances pheromone strength and heuristic desirability.
3) **Pheromone Update**:
   - Apply pheromone evaporation using Equation 3:

$$\tau_{ij} = (1 - \rho)\tau_{ij} \tag{3}$$

   - Deposit new pheromones based on the quality of the tours using Equation 4:

$$\tau_{ij} = \tau_{ij} + \frac{Q}{L} \tag{4}$$

   where L is the length of the tour.
4) **Iteration and Convergence**:
   - Repeat the tour construction and pheromone update steps for a predefined number of iterations.
   - Track and store the best solution found throughout the iterations.

B. Implementation Details

The implementation of the ACO algorithm, along with GA and SQP, was conducted using Python in a Google Colab environment. The complete code is available at the following link:

Google Colab Notebook

C. Algorithm Design

- **Import Libraries and Setup**: Utilized essential Python libraries such as NumPy, SciPy, Plotly, Matplotlib, and scikit-learn for computations and visualizations.
- **Define Helper Functions**: Functions for distance calculations, pheromone updates, and other utility operations were defined to streamline the algorithms.
- **Implement Core Algorithms**: Developed implementations for ACO, GA, and SQP to solve the TSP.
- **Develop Visualization Tools**: Created tools for generating 3D plots and performance metrics to visualize solutions and algorithm performance.
- **Conduct Parameter Sensitivity Analysis**: Analyzed how varying key parameters affect the performance and solution quality of the ACO algorithm.
- **Validate Solutions**: Ensured that all solutions met TSP constraints, such as visiting each city exactly once and forming a complete tour.

D. Comparison with Other Techniques

The performance of ACO was compared against Genetic Algorithms (GA) and Sequential Quadratic Programming (SQP) to evaluate solution quality, computational efficiency, and convergence behavior. These comparisons were based on analytical assessments and empirical results obtained from the implementations.

IV. RESULTS & ANALYSIS

A. Experimental Setup

- **Problem Instance**: 20 cities in 3D space, randomly generated to simulate a realistic TSP scenario.
- **Software Environment**:
  - Python 3.11.7
  - NumPy, SciPy, Plotly
  - Matplotlib, scikit-learn
- **Algorithm Parameters**:
  - **ACO**: Number of ants = 10, Iterations = 100, α = 1.0, β = 2.0, ρ = 0.5, Q = 100.

- **GA**: Population size = 50, Generations = 100, Mutation rate = 0.01.
- **SQP**: Initial solution derived from the Nearest Neighbor heuristic.

B. Algorithm Performance

TABLE I: Algorithm Performance Metrics

Algorithm	Path Length	Time (s)	Validation
ACO	657.56	2.501	Valid
GA	944.92	0.181	Valid
SQP	694.36	0.010	Valid

1) Individual Results: Table I summarizes the performance metrics of the three algorithms. ACO achieved the shortest path length, indicating its effectiveness in finding optimal solutions. GA and SQP, while faster, provided longer paths, reflecting a trade-off between solution quality and computational efficiency.

2) Comparative Analysis:

a) Solution Quality:

- **ACO** achieves the shortest path (657.56), indicating its effectiveness in finding optimal solutions.
- **SQP** provides a moderate solution (694.36), balancing quality and computational effort.
- **GA** shows a higher path length (944.92), suggesting room for improvement in convergence.

b) Computational Efficiency:

- **SQP**: Fastest execution (0.010s), suitable for quick approximations.
- **GA**: Quick execution (0.181s), offering a balance between speed and solution quality.
- **ACO**: Longest runtime (2.501s), justified by its superior solution quality.

c) Statistical Analysis:

- **Mean Path Lengths:** Calculated across multiple runs to assess average performance.
- **Standard Deviations:** Measured to understand the variability in solutions.
- **Improvement Percentages:** Evaluated to quantify the enhancements achieved by each algorithm over iterations.

d) Discussion: The comparative analysis indicates that ACO outperforms GA and SQP in terms of solution quality, albeit at the cost of longer computational times. SQP excels in speed but offers moderate solution quality, while GA provides a middle ground. These findings suggest that ACO is preferable when solution quality is paramount, whereas SQP is suitable for applications requiring rapid approximations.

C. Parameter Sensitivity

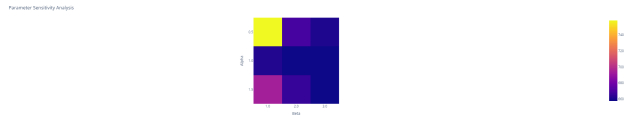

Fig. 1: ACO Parameter Sensitivity Analysis

The parameter sensitivity analysis in Figure 1 illustrates how varying the parameters α and β impacts the ACO algorithm's performance. Optimal parameter settings are crucial for balancing pheromone influence and heuristic desirability, thereby enhancing solution quality and convergence speed.

D. Solution Validation

All algorithms produced valid solutions with:

- **Complete Tours**: Each solution formed a complete tour, visiting all cities exactly once before returning to the start.
- **All Cities Visited**: Ensured that no city was omitted or visited multiple times.
- **Feasible Connections**: Maintained feasible connections between cities, adhering to TSP constraints.

This validation confirms that the implemented algorithms effectively solve the TSP without violating any fundamental constraints.

E. Visualization of Results

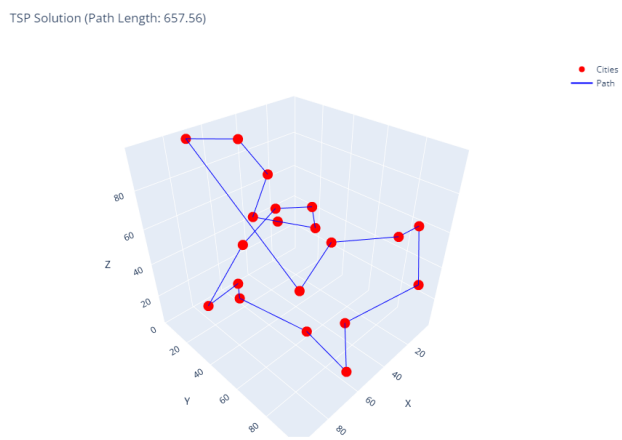

Fig. 2: ACO Solution Path

1) ACO Solution Path: Figure 2 depicts the solution path generated by the ACO algorithm, showcasing the optimal route identified across all cities.

Fig. 3: Comparative Convergence of ACO, GA, and SQP

2) Comparative Convergence: Figure 3 illustrates the convergence behavior of ACO in comparison with GA and

SQP. The graph highlights ACO's superior performance in achieving lower path lengths over iterations, despite its longer runtime.

V. CONCLUSIONS

A. Summary of Findings

- **ACO** demonstrates superior solution quality (657.56) despite longer runtime, making it highly effective for solving TSP.
- **SQP** provides efficient approximation (694.36) with minimal computation time (0.010s), suitable for scenarios requiring quick solutions.
- **GA** shows moderate performance (944.92) with reasonable execution time (0.181s), balancing between speed and solution quality.

B. Theoretical Implications

Results validate theoretical complexity analysis:

- **ACO's** $\mathcal{O}(n^2m)$ reflects higher computational costs, which are justified by its ability to find superior solutions.
- **SQP's** $\mathcal{O}(n^3)$ allows for quick local optimization, making it efficient for smaller problem instances.
- **GA's** $\mathcal{O}(gps^2)$ balances exploration and exploitation, providing a versatile approach for various optimization scenarios.

C. Practical Implications

- **Algorithm Selection Criteria**:
  - **ACO**: Preferred when solution quality is paramount, especially for large and complex TSP instances.
  - **SQP**: Ideal for applications requiring rapid approximations with limited computational resources.
  - **GA**: Suitable for problems where a balance between solution quality and computational efficiency is needed.
- **Implementation Considerations**:
  - **Parameter Tuning**: Critical for ACO to optimize the balance between pheromone influence and heuristic information.
  - **Population Diversity**: Important for GA to prevent premature convergence and ensure exploration of the solution space.
  - **Initial Solution Impact**: Influences the performance of SQP, highlighting the need for a good starting point.

D. Future Work

- **Hybrid Approaches**: Combining ACO with SQP to leverage the strengths of both algorithms.
- **Parallel ACO Implementation**: Enhancing computational efficiency by parallelizing the ACO algorithm.
- **Adaptive Parameter Tuning**: Developing methods for dynamic adjustment of ACO parameters during iterations.
- **Testing on Larger Instances**: Extending the analysis to more extensive TSP instances to evaluate scalability.
- **Multi-Objective Optimization**: Exploring solutions that optimize multiple objectives, such as minimizing time and cost simultaneously.

ACKNOWLEDGMENT

The authors would like to thank Dr. Ahmed Abdelsamea for his guidance and support throughout this course.

REFERENCES

[1] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.

[2] G. Laporte, "The traveling salesman problem: Overview of algorithms," *European Journal of Operational Research*, vol. 59, no. 2, pp. 231–247, 1992.

[3] M. Dorigo and T. Stützle, *Ant Colony Optimization*, Cambridge, MA: MIT Press, 2004.